Data Structures and Algorithmic Problem Solving Project

Project Title: PC Point - Online Computer Sales and Accessories

Student: Aaron Joseph | Register No: 26MCAB01 | Class: MCA B | Dept: KJIT
Course: Data Structures and Algorithmic Problem Solving | Year: 2026 - 27
Faculty Guide: Dr. Abinaya S

1. Abstract

PC Point is a web-based application for browsing and purchasing computer hardware, built to demonstrate core data structures and algorithms in a store management scenario. Linear structures (Linked List, LIFO Stack, FIFO Queue) handle dynamic sales records, expression-based transaction billing, and customer service queues. Hardware component relationships are explored using a Graph with Breadth-First Search (BFS). Product discovery uses Linear Search, Binary Search, and Merge Sort, while budget accessory selection uses a Greedy algorithm.

2. Introduction

- Background: Computer shops handle stock tracking, billing, queues, and budget selection.
- Problem Identified: Manual sales management causes slow lookups and unorganized records.
- Need for System: Lightweight app to store sales/queue data and search inventory fast.
- Proposed Solution: Flask web app backed by SQLite utilizing Linked Lists, LIFO Stacks, FIFO Queues, Graphs with BFS, Search/Sort algorithms, and Greedy selection.

3. Problem Statement

Small computer accessory shops lack structured methods for catalog search, sales recording, customer queueing, transaction billing evaluation, and budget recommendations.

4. Objectives

1. Implement Linked List, LIFO Stack, and FIFO Queue for sales, billing, and queues.
2. Apply Graph structures and Breadth-First Search (BFS) for component relationships.
3. Implement Linear Search, Binary Search, and Merge Sort for product discovery.
4. Solve budget-based accessory selection using a Greedy algorithm.
5. Integrate all data structures and algorithms into a presentable web application.

5. Proposed System

PC Point features HTML/CSS/JS frontend, Python Flask backend, and SQLite database.
Customer side: Browse catalog, search by ID/name, sort by price, explore graph, recommend.
Management side: Record sales in Linked List, evaluate bills via LIFO Stack, process Queue.

6. Data Structures and Algorithms to be Used

A. Linear Data Structures

- Linked List: Records sales transaction history dynamically.
- Stack using Linked List: Billing Expression Evaluator ((price * qty) + tax - discount).
- Queue using Linked List: Customer service line management (FIFO).

B. Graph Structures

- Graph (Adjacency List): Hardware component and category relationships.
- Breadth-First Search (BFS): Level-by-level component graph traversal.

C. Searching and Sorting

- Linear Search: Search products by text (brand/category).
- Binary Search: Search products by ID in O(log n) time.
- Merge Sort: Sort listings by price, name, or ID in O(n log n) divide-and-conquer time.

D. Algorithmic Problem Solving

- Greedy Technique: Budget-based accessory selection algorithm.

7. Module Description

Module 1: Dynamic Sales History Management (Singly Linked List)
Module 2: Step-by-step Bill Calculation & Customer Support Desk (LIFO Stack / FIFO Queue)
Module 3: Hardware Component Relationships (Graph & BFS Traversal)
Module 4: Product Discovery & Sorting (Linear/Binary Search + Merge Sort)
Module 5: Budget Accessory Selection (Greedy Algorithm)

8. Tools and Technologies

- Programming Languages: Python (backend logic), JavaScript (interactivity), HTML & CSS
- Framework & Database: Flask, SQLite
- IDE: Visual Studio Code

9. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., Introduction to Algorithms, MIT Press.
- Python Software Foundation, Python Documentation, https://docs.python.org
- Flask Documentation, https://flask.palletsprojects.com